

Lab 3: Component Modelling & Architectural Pattern Selection

Software Engineering Lab — Submission Report

Name	Arnav Chaudhary
Section	B
SRN	PES1UG24CS081
Project	Personalized Meal & Diet Subscription Manager
Problem Statement	#19 — Personalized Meal & Diet Subscription Manager (Healthcare & Telemedicine)
GitHub Repository	https://github.com/ArnavChaudhary21/SE-LABS-PES1UG24CS081

1. Component Diagram

The diagram below decomposes the Personalized Meal & Diet Subscription Manager (from Lab 1) into replaceable, independently deployable components. It follows UML component notation (<<component>> stereotype, lollipop-style provided interfaces implied at each box) grouped into a Presentation layer, an Application Service layer, a Data layer, and External systems, connected through an Event Bus for asynchronous workflow steps (e.g. draft → review → approval → release).

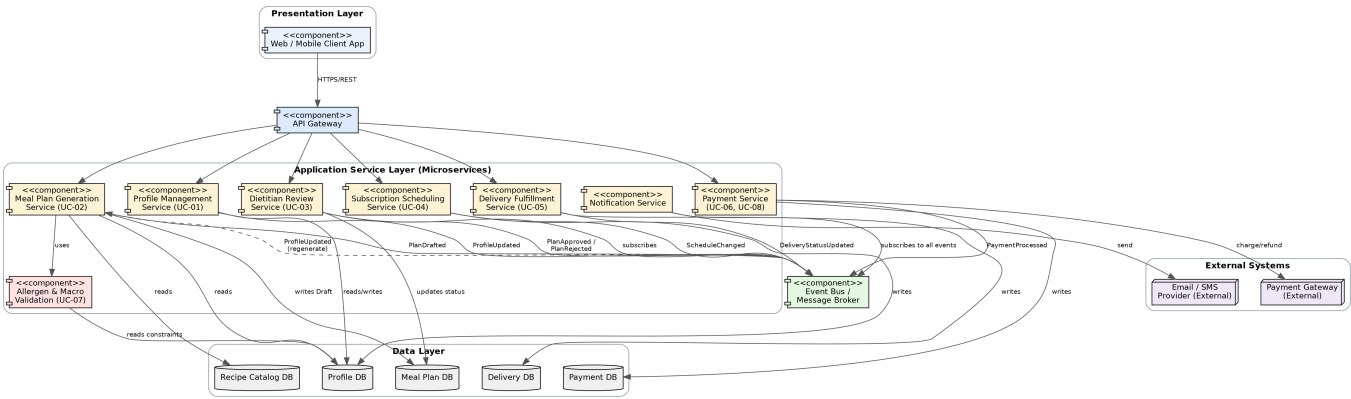


Figure 1: Component diagram for the Meal & Diet Subscription Manager.

1.1 Component Responsibilities

Component	Responsibility	Maps to
Web / Mobile Client App	Presentation layer; subscriber and dietitian UI.	All use cases
API Gateway	Single entry point; routing, auth, rate limiting.	Cross-cutting
Profile Management Service	CRUD on dietary profile (allergies, diabetic limits, calorie goals).	UC-01
Meal Plan Generation Service	Composes the 7-day menu; orchestrates validation; sets Draft status.	UC-02

Component	Responsibility	Maps to
Allergen & Macro Validation	Filters recipe catalog against a subscriber's constraints.	UC-07 (include)
Dietitian Review Service	Approve/reject workflow; escalation after repeated rejection.	UC-03
Subscription Scheduling Service	Pause/skip/reschedule up to the 12-hour cutoff.	UC-04, NFR-001
Delivery Fulfillment Service	Tracks Delivered/Missed/Rescheduled status per order.	UC-05
Payment Service	Charges/refunds via the external Payment Gateway; discount codes.	UC-06, UC-08
Notification Service	Sends reminders and status alerts via email/SMS.	FR-005
Event Bus / Message Broker	Decouples services; carries domain events (PlanDrafted, PlanApproved, etc.).	Cross-cutting

2. Architectural Pattern Selection & Justification

2.1 Candidate Patterns Considered

Pattern	Fit for this system
Layered (Monolithic) Architecture	Simple to build, but couples the Dietitian-review workflow, payments, and delivery tracking into one deployable unit. A spike in meal-plan generation load (NFR-001) would force scaling the entire application, and a bug in one module risks the whole system's availability.
Pure Client-Server	Works for basic CRUD (e.g. profile edits) but has no natural place for asynchronous, multi-step workflows such as Draft → Dietitian Review → Approved → Release, or for fan-out notifications triggered by a single profile update.
Microservices + Event-Driven (with API Gateway)	Each bounded context (profile, plan generation, review, scheduling, delivery, payment, notification) is isolated, independently scalable, and independently deployable; asynchronous events naturally model the review/approval workflow and the 'regenerate on profile update' rule.

2.2 Selected Pattern: Microservices with Event-Driven Communication, fronted by an API Gateway

The system adopts a **microservices architecture** where each Lab 1 use case maps to one primary service, communicating synchronously through a shared **API Gateway** for client-facing requests and asynchronously through an **Event Bus** for cross-service workflow steps. This hybrid was chosen for the following reasons, each tied directly to the Lab 1 requirements table:

- **Clinical workflow fit (FR-002):** A weekly plan must move Draft → Approved → Released only through a Dietitian action. Modelling this as domain events (PlanDrafted, PlanApproved, PlanRejected) published on the Event Bus keeps the Meal Plan Generation Service and Dietitian Review Service decoupled — the generation service does not need to know who reviews the plan or how, only that a PlanApproved event will eventually arrive.

- **Reactive profile updates (FR-003):** When Profile Management Service emits ProfileUpdated, the Meal Plan Generation Service subscribes and regenerates the next pending plan automatically, satisfying the 'one menu-generation cycle' acceptance criterion without tight coupling between the two services.
- **Independent scalability (NFR-001):** Meal-plan generation, dietitian review, and payment processing have very different load profiles (generation runs in a weekly batch burst; payments trickle in continuously). Microservices let each be scaled and load-tested independently to meet the 12-hour modification cutoff under peak load.
- **Security & data isolation (NFR-002):** Splitting Profile Management and Payment Service into separate components with their own databases lets encryption, access control, and audit logging be scoped tightly to health data versus financial data, rather than one large schema with a single access-control surface.
- **Extensibility & failure isolation:** Notification Service, Delivery Fulfillment Service, and Payment Service can fail or be redeployed without taking down meal-plan generation or the review workflow — directly supporting the Lab 1 postcondition that a fallback safe-menu remains active if generation fails.
- **API Gateway as a single entry point:** Centralises authentication and role checks (Subscriber vs Dietitian), so 'restrict access to authenticated Subscriber and Dietitian roles only' (NFR-002) is enforced consistently before a request reaches any service.

2.3 Trade-offs Accepted

Microservices introduce operational overhead: distributed tracing/monitoring is needed across services, eventual consistency must be handled (e.g. a subscriber may briefly see a 'Draft' plan status before review completes), and the Event Bus becomes a critical piece of infrastructure that itself needs high availability. For a system handling health-related and financial data, this overhead is accepted in exchange for the isolation, auditability, and independent scaling the requirements demand.

2.4 Mapping Summary

Requirement	How the architecture satisfies it
FR-001 (allergen/macro safety)	Allergen & Macro Validation isolated as its own component, invoked synchronously (include) so a plan can never be drafted without passing validation.
FR-002 (dietitian approval gate)	Enforced via the Draft/Approved event contract between Meal Plan Generation and Dietitian Review services.
FR-003 (profile-driven regeneration)	ProfileUpdated event subscription on Meal Plan Generation Service.
FR-004 / FR-005 (delivery status, reminders)	Delivery Fulfillment and Notification services, decoupled via the Event Bus so a notification failure cannot block delivery tracking.
NFR-001 (12-hour modification cutoff, load)	Subscription Scheduling Service is independently scalable and only interacts with other services asynchronously.
NFR-002 (encryption & role-restricted access)	API Gateway enforces authentication/roles; Profile and Payment services keep separate, independently-encrypted data stores.